

Bàn sơ về ứng dụng của tư tưởng nhân đôi trong thi tin học

Zhu Chenguang
Trường THPT số 1 Vu Hồ, An Huy

2005

Nguồn gốc: Bàn sơ về ứng dụng của tư tưởng nhân đôi trong thi tin học

IOI China candidate team papers / 2005 / Zhu Chenguang.doc

Tóm tắt

Tư tưởng nhân đôi là một cách nghĩ độc đáo và khéo léo để giải các bài toán tin học. Bài viết phân tích hai hướng ứng dụng của tư tưởng này trong thi tin học. Phần đầu nêu vai trò và cách dùng cơ bản của nhân đôi; phần thứ hai trình bày cách tăng tốc chuyển trạng thái khi quy luật biến đổi không đổi; phần thứ ba trình bày cách tăng tốc thao tác trên đoạn; phần thứ tư thảo luận vì sao ta thường mở rộng phạm vi gấp đôi mỗi lần thay vì gấp ba hay nhiều hơn; phần cuối tổng kết lại ý nghĩa và cách vận dụng linh hoạt tư tưởng nhân đôi.

Từ khóa: tư tưởng nhân đôi; quy luật biến đổi; chuyển trạng thái; thao tác trên đoạn.

1 Dẫn nhập

Nhân đôi là một tư tưởng rất tinh tế và được dùng rộng rãi trong các kỳ thi tin học. Tuy có thể xuất hiện trong nhiều hoàn cảnh khác nhau, bản chất chung của nó là: dựa trên thông tin đã tính được, mỗi lần mở rộng phạm vi đang xét lên gấp đôi để tăng tốc thao tác. Sắp xếp trộn quen thuộc thực ra cũng là một ứng dụng kinh điển của tư tưởng nhân đôi.

Trong giải bài tập tin học, tư tưởng nhân đôi thường có hai hướng ứng dụng chính:

- tăng tốc chuyển trạng thái khi quy luật biến đổi giống nhau;
- tăng tốc thao tác trên đoạn.

Hai phần tiếp theo sẽ lần lượt thảo luận hai hướng này.

2 Ứng dụng thứ nhất: tăng tốc chuyển trạng thái

Trước hết xét một ví dụ rất đơn giản: biết số thực a , tính a^{17} .

Cách trực tiếp là đặt $b = a$, rồi lặp 16 lần phép gán $b = b \cdot a$. Như vậy cần 16 phép nhân. Một cách khác là tính

$$a_0 = a, \quad a_1 = a^2, \quad a_2 = a^4, \quad a_3 = a^8, \quad a_4 = a^{16},$$

trong đó

$$a_i = a_{i-1}^2 \quad (1 \leq i \leq 4).$$

Để có $a_0, \dots, a_4$ chỉ cần 4 phép nhân, rồi

$$a^{17} = a \cdot a^{16} = a_0 a_4,$$

nên tổng cộng chỉ cần 5 phép nhân.

Mở rộng ra, giả sử cần tính a^n . Viết n dưới dạng nhị phân, tức là lấy những bit khác 0:

$$n = 2^{b_1} + 2^{b_2} + \dots + 2^{b_w}, \quad b_1 < b_2 < \dots < b_w.$$

Từ a ta lần lượt bình phương để có

$$a^{2^0}, a^{2^1}, a^{2^2}, \dots, a^{2^{b_w}}.$$

Theo quy tắc lũy thừa,

$$a^n = a^{2^{b_1}} a^{2^{b_2}} \dots a^{2^{b_w}}.$$

Vì biểu diễn nhị phân của n có không quá $\lceil \log_2 n \rceil + 1$ bit khác 0, số phép nhân chỉ còn $\mathcal{O}(\log n)$ thay vì $\mathcal{O}(n)$. Trong cài đặt, không cần lưu toàn bộ dãy lũy thừa; khi duyệt các bit từ thấp lên cao, ta vừa bình phương vừa nhân vào kết quả nếu bit hiện tại bằng 1.

Thuật toán lũy thừa nhanh giảm số phép nhân vì nó biến việc tính a^n thành việc nhân một số ít lũy thừa có số mũ là lũy thừa của 2. Mỗi phần tử sau trong dãy

$$a, a^2, a^4, a^8, \dots$$

được tính từ phần tử trước bằng đúng một phép bình phương, trong khi cách thô cần nhiều phép nhân lặp lại.

Trong thực tế, a có thể là số thực, ma trận, hoặc một trạng thái trừu tượng; quy luật biến đổi cũng có thể là phép nhân ma trận, chuyển trạng thái trong quy hoạch động, v.v. Chỉ cần thỏa hai điều kiện sau, ta có thể dùng tư tưởng nhân đôi:

1. quy luật biến đổi ở mỗi bước là giống nhau;
2. phép ghép các quy luật biến đổi thỏa tính kết hợp.

Trong ví dụ lũy thừa, quy luật là phép nhân, và phép nhân thỏa tính kết hợp.

2.1 Ví dụ: chuyển động của xúc xắc

Mô tả bài toán. Cho một xúc xắc sáu mặt; mỗi mặt có một trọng số nguyên từ 1 tới 50. Xúc xắc di chuyển trên một dải rộng 4 hàng và kéo dài vô hạn sang trái, sang phải. Tọa độ cột là các số nguyên

$$\dots, -3, -2, -1, 0, 1, 2, 3, \dots,$$

còn tọa độ hàng là 1, 2, 3, 4. Mỗi ô vừa khít một mặt của xúc xắc. Mỗi bước, xúc xắc có thể lăn sang trước, sau, trái hoặc phải một ô, nhưng không được ra khỏi dải. Chi phí của một bước là trọng số của mặt hướng lên sau khi lăn.

Biết vị trí hiện tại và hướng các mặt của xúc xắc, hãy tìm chi phí nhỏ nhất để di chuyển tới một vị trí mới. Trị tuyệt đối của tọa độ cột có thể tới 10^9 .

Phân tích. Nếu không xét sự chênh lệch cột quá lớn, bài toán có thể giải bằng quy hoạch động. Mỗi ô được tách thành 24 trạng thái theo hướng của xúc xắc, nên mỗi cột có $24 \cdot 4 = 96$ trạng thái. Ta có thể coi bài toán là đồ thị phân tầng: từ 96 trạng thái của cột này chuyển sang 96 trạng thái của cột kế tiếp, rồi tiếp tục tới cột đích. Mỗi lần chuyển giữa hai cột kề nhau có thể dùng Dijkstra để tính chi phí nhỏ nhất

giữa các trạng thái. Nhưng độ phức tạp tỉ lệ với độ lệch cột n , trong khi n có thể tới 10^9 , nên cách này không đủ.

Ta kiểm tra hai điều kiện để dùng nhân đôi. Nếu xúc xắc đi từ trạng thái A ở cột 1 tới trạng thái B ở cột 2 với chi phí c , thì từ trạng thái tương ứng A ở cột 2 tới trạng thái B ở cột 3 cũng có chi phí c . Tổng quát hơn, nếu từ cột i ở trạng thái A tới cột $i + k$ ở trạng thái B mất chi phí c , thì từ cột j ở trạng thái A tới cột $j + k$ ở trạng thái B cũng mất chi phí c . Đây là quy luật biến đổi giống nhau.

Mặt khác, quy hoạch động có tính chất cấu trúc con tối ưu. Nếu đi từ A tới C qua B , thì chi phí nhỏ nhất của toàn đường bằng tổng chi phí nhỏ nhất của các đoạn tương ứng, và thứ tự ghép các quy luật chuyển có thể xem là kết hợp. Do đó có thể dùng nhân đôi.

Giả sử đích nằm bên phải điểm xuất phát. Trước hết dùng Dijkstra để tính ma trận chuyển M_1 kích thước 96×96 , trong đó $M_1[A, B]$ là chi phí nhỏ nhất để đi từ trạng thái A của một cột sang trạng thái B của cột ngay bên phải. Từ M_1 tính được M_2 , chi phí đi xa 2 cột; rồi $M_4, M_8, \dots$. Phép ghép hai ma trận chuyển là phép nhân trong đại số min-plus:

$$(X \circ Y)[i, j] = \min_k \{X[i, k] + Y[k, j]\}.$$

Nếu độ lệch cột là

$$n = 2^{b_1} + 2^{b_2} + \dots + 2^{b_w},$$

ta xuất phát từ vector trạng thái ban đầu và lần lượt áp dụng các ma trận $M_{2^{b_1}}, M_{2^{b_2}}, \dots, M_{2^{b_w}}$. Kết quả là chi phí nhỏ nhất từ điểm xuất phát tới mọi trạng thái của cột đích.

Dijkstra ở bước đầu có thể xem là hằng số vì số trạng thái trong một số cột cần xét là hằng. Mỗi lần nhân đôi ma trận chuyển mất khoảng 96^3 , và số lần nhân đôi là $\log_2 n$, nên độ phức tạp là

$$\mathcal{O}(96^3 \log n).$$

Ví dụ này làm rõ một điểm quan trọng: đối tượng mà nhân đôi tác động thực ra là **quy luật biến đổi**, không phải trạng thái cụ thể. Nhân đôi tính ra “lượng biến đổi” từ một trạng thái tới một trạng thái khác. Nếu trạng thái ban đầu là A , trạng thái cuối là B , và lượng biến đổi là c , kết quả có thể viết trừu tượng là $A * c$, trong đó dấu $*$ biểu thị cách áp dụng quy luật biến đổi.

Một khuôn giả mã tổng quát là:

```
DoublingAlgorithm( $A, n$ ) :
   $b \leftarrow a, \quad c \leftarrow$  quy luật đơn vị;
  while  $n > 0$  do
    if  $n$  lẻ then  $c \leftarrow c \circ b$ ;
     $b \leftarrow b \circ b$ ;
     $n \leftarrow \lfloor n/2 \rfloor$ ;
  return  $A * c$ .
```

Ở đây a, b, c đều là quy luật biến đổi; $b \circ b$ biểu thị quy luật tương đương với việc áp dụng b hai lần liên tiếp.

Cần nhấn mạnh lại: quy luật biến đổi phải giống nhau và phép ghép phải thỏa tính kết hợp. Phép chia không thỏa điều kiện này nếu giữ nguyên dạng:

$$a_1/a_2/a_3/\dots/a_n \neq a_1/(a_2/a_3/\dots/a_n).$$

Muốn dùng nhân đôi, ta phải chuyển nó thành một phép hợp lệ, hoặc tìm phương pháp khác.

3 Ứng dụng thứ hai: tăng tốc thao tác trên đoạn

Trong thao tác trên đoạn có nhiều cấu trúc mạnh, tiêu biểu là cây đoạn. Tư tưởng nhân đôi cũng xử lý được nhiều bài toán đoạn, và trong một số trường hợp đặc biệt còn rất gọn.

3.1 Khuôn xử lý đoạn bằng nhân đôi

Tiền xử lý. Với mỗi vị trí A , lưu tính chất của các đoạn

$$[A, A + 2^0 - 1], [A, A + 2^1 - 1], [A, A + 2^2 - 1], \dots, [A, A + 2^k - 1],$$

trong đó 2^k không vượt quá độ dài cần xét. Khi tính tính chất của đoạn $[A, A + 2^{i+1} - 1]$, ta ghép hai kết quả đã biết:

$$[A, A + 2^i - 1] \quad \text{và} \quad [A + 2^i, A + 2^{i+1} - 1].$$

Trả lời truy vấn. Với một đoạn $[A, B]$, có hai cách lấy kết quả:

- Nếu việc chồng lấn đoạn không ảnh hưởng tới kết quả, chẳng hạn lấy min hoặc max, đặt

$$k = \lfloor \log_2(B - A + 1) \rfloor.$$

Kết quả của $[A, B]$ được ghép từ hai đoạn

$$[A, A + 2^k - 1] \quad \text{và} \quad [B - 2^k + 1, B].$$

Hai đoạn có thể chồng lên nhau nhưng không làm sai kết quả. Thời gian trả lời là $\mathcal{O}(1)$.

- Nếu chồng lấn làm thay đổi kết quả, chẳng hạn bài đếm, ta phân rã $[A, B]$ thành các đoạn có độ dài là lũy thừa của 2 theo biểu diễn nhị phân. Mỗi lần lấy đoạn lớn nhất có thể, nên số đoạn không quá $\lfloor \log_2 N \rfloor + 1$. Thời gian trả lời là $\mathcal{O}(\log N)$.

Tổng quát, tiền xử lý mất $\mathcal{O}(N \log N)$ thời gian và bộ nhớ $\mathcal{O}(N \log N)$; mỗi truy vấn mất $\mathcal{O}(1)$ hoặc $\mathcal{O}(\log N)$ tùy bài.

3.2 Ví dụ: RMQ tổng quát

Mô tả bài toán. Cho mảng A độ dài n . Mỗi truy vấn (i, j) với $i \leq j$ yêu cầu tìm chỉ số của phần tử nhỏ nhất trong $A[i..j]$.

Phân tích. Dựng bảng B , trong đó $B[i, k]$ là chỉ số của phần tử nhỏ nhất trong đoạn

$$A[i..i + 2^k - 1].$$

Theo khuôn trên, bảng này được tính trong $\mathcal{O}(N \log N)$. Với truy vấn (i, j) , đặt

$$k = \lfloor \log_2(j - i + 1) \rfloor.$$

Ta chỉ cần so sánh

$$A[B[i, k]] \quad \text{và} \quad A[B[j - 2^k + 1, k]]$$

để lấy chỉ số nhỏ hơn. Vì phép min không bị ảnh hưởng bởi chồng lấn, thời gian trả lời là $\mathcal{O}(1)$. Đây là một ứng dụng rất gọn của nhân đôi trong giảm độ phức tạp.

3.3 Ví dụ: tổ tiên chung gần nhất trên cây

Mô tả bài toán. Cho một cây có n đỉnh. Mỗi truy vấn hỏi tổ tiên chung gần nhất của hai đỉnh i, j ; số truy vấn là m .

Phân tích. Nếu mỗi lần tìm trực tiếp, độ phức tạp có thể tới $\mathcal{O}(MN)$. Dùng nhân đôi, với mỗi đỉnh A ta lưu:

A , cha của A , tổ tiên cách 2^1 cạnh, tổ tiên cách 2^2 cạnh, ...

Nếu xử lý các đỉnh theo thứ tự duyệt trước, ta có thể dùng kết quả của tổ tiên đã tính trước để điền bảng. Tiền xử lý mất $\mathcal{O}(N \log N)$.

Để trả lời truy vấn A, B , mục tiêu là tìm đỉnh cùng xuất hiện trong hai dãy tổ tiên với độ sâu lớn nhất. Một cách chuẩn là:

1. Đưa đỉnh sâu hơn lên cùng độ sâu với đỉnh còn lại bằng các bước nhảy 2^k .
2. Nếu hai đỉnh đã trùng nhau, đó là LCA.
3. Ngược lại, từ k lớn xuống nhỏ, nếu tổ tiên 2^k của hai đỉnh khác nhau thì nâng cả hai lên mức đó.
4. Cha của hai đỉnh sau bước trên là LCA.

Mỗi truy vấn mất $\mathcal{O}(\log N)$.

Bản gốc trình bày một biến thể nhị phân theo dãy tổ tiên của một đỉnh và dùng bảng nhân đôi để tìm tổ tiên cùng tầng của đỉnh còn lại. Dù cách viết khác nhau, bản chất vẫn là tiền xử lý các bước nhảy lũy thừa của 2 và dùng chúng để thu hẹp khoảng tìm kiếm.

Tài liệu tham khảo của tác giả còn nêu cách chuyển LCA về bài toán RMQ ± 1 , có thể đạt tiền xử lý $\mathcal{O}(N)$ và trả lời $\mathcal{O}(1)$, nhưng cài đặt phức tạp hơn nên bài viết không triển khai.

Hai ví dụ trên cho thấy nhân đôi có thể xử lý thao tác trên đoạn. Trong RMQ và LCA, nhân đôi chưa hẳn luôn là phương án mạnh nhất về mặt lý thuyết, nhưng cài đặt đơn giản, dễ nghĩ và rất hữu dụng. Khi xây dựng suffix array, nhân đôi còn đóng vai trò cực kỳ quan trọng.

Một điểm đáng chú ý là trong bài LCA và suffix array, đoạn không phải lúc nào cũng lộ rõ trong đề. Nhiều khi đoạn là cấu trúc do ta tự xây dựng. Điều này cho thấy nhân đôi là một tư tưởng, không phải một khuôn cứng nhắc; thí sinh cần dựa vào tư tưởng ấy để tạo ra thuật toán khéo léo và hiệu quả.

4 Một thảo luận thú vị

Bản chất của nhân đôi là mỗi lần mở rộng phạm vi lên gấp đôi. Vậy có thể mỗi lần mở rộng gấp ba, gấp bốn hoặc nhiều hơn không?

Gọi cách mỗi lần mở rộng theo cơ số k là “ k -tăng”, với $k \geq 2$. Tư tưởng nhân đôi thông thường chính là trường hợp $k = 2$. Với cơ số k , số lần mở rộng tối đa xấp xỉ $\log_k N$. Nhưng để mở rộng phạm vi lên k lần, nếu không tiếp tục dùng nhân đôi bên trong, cần khoảng $k - 1$ thao tác ghép. Do đó độ phức tạp tương ứng là

$$\mathcal{O}((k - 1) \log_k N).$$

Khi $k = 2$, giá trị này là $\log_2 N$. So sánh với $k \geq 3$, ta cần xét

$$(k - 1) \log_k N \quad \text{và} \quad \log_2 N.$$

Vì

$$\log_k N = \frac{\log_2 N}{\log_2 k},$$

nên tỉ số là

$$\frac{(k-1) \log_k N}{\log_2 N} = \frac{k-1}{\log_2 k}.$$

Với $k = 2$ tỉ số bằng 1. Với $k \geq 3$, ta có

$$k-1 > \log_2 k,$$

nên tỉ số lớn hơn 1. Nói cách khác,

$$(k-1) \log_k N > \log_2 N \quad (k \geq 3).$$

Điều này giải thích từ góc độ toán học vì sao mở rộng gấp đôi là lựa chọn hiệu quả tự nhiên.

5 Kết luận

Bài viết đã giới thiệu ngắn gọn vai trò của tư tưởng nhân đôi và hai hướng ứng dụng chính: tăng tốc chuyển trạng thái khi quy luật biến đổi giống nhau, và tăng tốc thao tác trên đoạn. Nguồn gốc hiệu quả của nhân đôi nằm ở việc tận dụng đúng mức các kết quả đã tính. Từ phân tích ở trên, có thể thấy nhân đôi gần như tận dụng việc tái sử dụng này tới cực hạn. Vì vậy các thuật toán được thiết kế theo tư tưởng nhân đôi thường ít phép tính dư thừa, vừa hiệu quả vừa gọn.

Nắm vững nhân đôi giúp ta tìm được lối đi riêng khi phân tích bài toán, nhanh chóng đạt được thuật toán có độ phức tạp thời gian, bộ nhớ và cài đặt tương đối thấp. Tuy bài viết đã nêu hai dạng ứng dụng và khuôn tổng quát, vai trò của tư tưởng nhân đôi không dừng ở đó; những biến hóa trong thực tế không thể gói hết trong một vài mô hình. Hy vọng bạn đọc từ đây cảm nhận được chiều sâu của tư tưởng nhân đôi, rồi trong quá trình luyện tập dần tích lũy kinh nghiệm, đưa nó tự nhiên vào cách suy nghĩ của mình.

Lời cảm ơn

Xin cảm ơn thầy Jiang Tao đã đọc bài và đưa ra những ý kiến sửa đổi quý báu. Xin cảm ơn huấn luyện viên Liu Rujia và bạn Xu Zhilei đã thảo luận sâu với tác giả về việc viết bài.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa, 2004.
2. Xu Zhilei, luận văn đội tuyển tập huấn IOI 2004, *Suffix Array*.
3. CEPC 2003 Problem D, *Dice Contest*.
4. USACO 2004 February Contest Green Problems.

A Ghi chú về phụ lục gốc

Bản gốc kèm phát biểu bài cho ví dụ xúc xắc, RMQ, bài USACO *Distance Queries*, cùng mã nguồn minh họa bằng C/C++. Theo quy ước của bản dịch này và theo yêu cầu không dịch code templates, phần mã nguồn không được chuyển ngữ. Các ý tưởng thuật toán của những chương trình đó đã được trình bày trong thân bài.

Farewell, My Friend! Bài xúc xắc cho sáu trọng số $f(1), \dots, f(6)$, vị trí đầu (x_1, y_1) và vị trí cuối (x_2, y_2) trên dải bốn hàng vô hạn theo chiều ngang. Ban đầu mặt 6 ở đáy, mặt 1 ở trên, mặt 2 hướng ra trước. Cần in chi phí nhỏ nhất để lăn tới đích.

RMQ tổng quát. Cho N số nguyên $A[1..N]$ và M cặp chỉ số (x, y) với $x \leq y$. Với mỗi cặp, in chỉ số của phần tử nhỏ nhất trong $A[x..y]$.

Distance Queries. Bài USACO cho một cây đường nông trại với các cạnh ngang dọc có độ dài. Sau khi đọc mô tả cây, cần trả lời nhiều truy vấn khoảng cách giữa hai nông trại, tức tổng độ dài các cạnh trên đường đi duy nhất nối chúng.